



Unit-1

Search in Linear Array

Searching in a linear array involves finding the position or value of a particular element within the array. There are several algorithms commonly used for searching in a linear array, including linear search and binary search. Let's first discuss linear search algorithm:

Linear Search

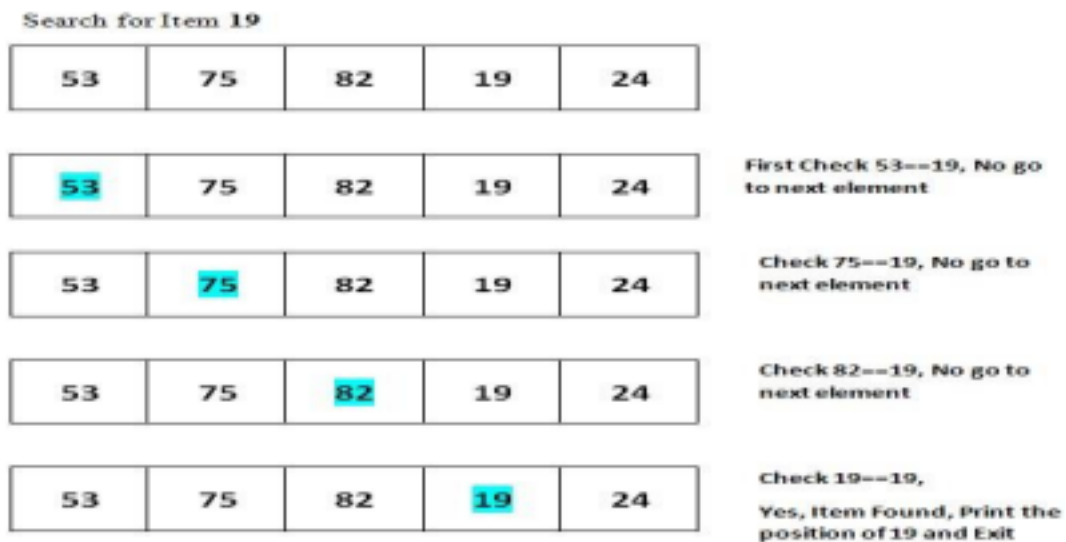
Linear search is a simple algorithm that checks each element in the array one by one until it finds the desired element or reaches the end of the array. It is suitable for unsorted arrays and small sized arrays.

Here's the step-by-step process of the linear search algorithm:

- 1) Start from the first element of the array or data structure.
- 2) Compare the current element with the target element you are searching for.
- 3) If the current element matches the target element, the search is successful, and you can return the index or position of the element.
- 4) If the current element does not match the target element, move to the next element in the structure.
- 5) Repeat steps 2-4 until either the target element is found or you reach the end of the structure.
- 6) If you reach the end of the structure without finding the target element, the search is unsuccessful, and you can return a "not found" indicator.

Example

Let's take an example for Linear Search: Array is given with elements as 53,75,82,19,24 respectively position at 1,2,3,4,5, an Item 19 is given to search in the above array. Searching process can be seen in the following diagram.



Algorithm of Linear Search

Linear Search (Data, N, ITEM, POS, FLAG)

Here Data is a Linear Array with size of N elements; ITEM is a given item needs to be searched. FLAG is used If ITEM is searching is successful or unsuccessful. The Objective of this algorithm is to find the position POS of ITEM in array DATA.

Step 1 [Initialize Counter] Set K:=LB and FLAG:=0

Step 2 Repeat Step 3 and 4 while K<= UB

Step 3 [Check Element with ITEM] check LA[K]=ITEM

Step 4 [ITEM matches? Yes] Set FLAG:=1 and POS:=K

Step 5 [ITEM matches? No, Increase Counter] Set K = K+1

(End of Loop)

Step 6 [FLAG is set to 1] Print “ITEM Found at Position POS”

Step 7 [FLAG is not set] Print “ITEM Not Found”.

Binary Search

Binary search is a more efficient algorithm for searching in **sorted arrays**. It works by repeatedly dividing the search space in half until the target element is found or the subarray is empty. This algorithm is significantly faster than linear search for large-sized sorted arrays. As this topic is covered in Unit one also, so we not going into deep.

Here's how the binary search algorithm works:

1. Start with a sorted array, where the lower bound is at index 1 and the upper bound is at index n, where n is the size of the array.
1. Compute the middle index as the floor of (lower bound + upper bound) / 2.
2. Compare the target value with the element at the middle index.
3. If the target value matches the middle element, the search is successful, and the middle index is returned.
4. If the target value is less than the middle element, discard the upper half of the array by updating the upper bound to (middle index - 1).
5. If the target value is greater than the middle element, discard the lower half of the array by updating the lower bound to (middle index + 1).
6. Repeat steps 2-6 until either the target value is found or the lower bound becomes greater than the upper bound.
7. If the target value is not found, the search is unsuccessful, and -1 is returned to

indicate this.

Implementation of Binary Search



**VIVEKANANDA GLOBAL
UNIVERSITY, JAIPUR**

(Established by Act 11/2012 of Rajasthan Govt. Covered u/s 2 (f) of UGC Act, 1956)

**NAAC
ACCREDITED
A+
UNIVERSITY**

Here;

lower bound = 1

upper bound = 7

Target value = 60

1. Start from list with above details
2. Now calculate mid index = $(1+7)/2 = 4$; value at 4 index or position is 40
3. So first check the target value (i.e. 60) at mid index value (i.e. 40);
4. if they matches the result will be successful and 4 position will be returned. But here they are not matching means adjust lower bound and upper bound accordingly. Here the value at target value $>$ mid index; so we need to search in the right side of the list i.e. 50, 60, 70; for getting this we need to shift the lower index to mid index +1.
5. [**Note** * if the target value $<$ mid index; in this case we need to search in the left side of list; i.e. 10, 20, 30; for getting this we need to shift the upper index to mid index -1]
6. Repeat steps 2-6 until either the target value is found or the lower bound becomes greater than the upper bound.

Binary search is a highly efficient algorithm with a time complexity of $O(\log n)$, where n is the number of elements in the sorted array. It significantly reduces the search space by half in each iteration, making it particularly effective for large arrays.

Insertion and Deletion in Linear Array

In a linear array, insertion and deletion operations involve adding or removing elements at specific positions within the array. The complexity of these operations can vary depending on the array implementation.



**VIVEKANANDA GLOBAL
UNIVERSITY, JAIPUR**

(Established by Act 11/2012 of Rajasthan Govt. Covered u/s 2 (f) of UGC Act, 1956)

**NAAC
ACCREDITED
A+
UNIVERSITY**

Inserting an element at the "end" of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element. Similarly Deletion of an element can also be easily done. On the other hand, suppose we need to insert an element in the middle of the array. Then, on the average, half of the elements must downward to new locations to handle the new element.

Let's discuss the common methods for insertion and deletion in a linear array:

Insertion in Linear Array:

- ✓ To insert an element into a linear array, you typically need to shift the existing elements to accommodate the new element. Here's the general process:
- ✓ Determine the position at which you want to insert the new element.
- ✓ If the array is already full, you may need to allocate additional memory or resize the array to create space for the new element.
- ✓ Shift the elements after the insertion position to the right by one index to make room for the new element.
- ✓ Insert the new element at the desired position.
- ✓ Update the length or size of the array if necessary.

Example

Suppose NAME is a List of country names; an 8-element linear array, and suppose five names are in the array, as in Fig. (a). Observe that the names are listed alphabetically, and suppose we want

to keep the array names alphabetical at all times. Suppose **Brazil** is added to the array. Then Denmark, Japan, Mexico and Turkey must each be moved downward one location, as in Fig. (b). Next suppose **India** is added to the array; then Japan, Mexico and Turkey must be moved, as in Fig. (c). Last, suppose Denmark is removed from the array. Then the five names India, Japan,



**VIVEKANANDA GLOBAL
UNIVERSITY, JAIPUR**

(Established by Act 11/2012 of Rajasthan Govt. Covered u/s 2 (f) of UGC Act, 1956)

**NAAC
ACCREDITED
A+
UNIVERSITY**

Mexico and Turkey must each be moved upward one location, as in Fig. (d). Clearly such movement of data would be very expensive if thousands of names were in the array.

Array NAME			
(a)Initial	(b)Inserting	(c)Inserting “Brazil”	Afghanistan
			Brazil
			India
			Japan
			Mexico
			Turkey
(d)Deleting “Denmark”			Afghanistan
	Brazil		
	India		
	Japan		
	Mexico		
	Turkey		

Algorithms for Insertion

The following algorithm inserts a data element ITEM into the Kth position in a linear array LA with N elements. The first four steps create space in LA by moving downward one location each element from the Kth position on. We emphasize that these elements are moved in reverse order i.e., first LA[N], then LA[N-1],..., and last LA[K]; otherwise data might be erased. In more detail, we first set J:= N and then, using J as a counter, decrease J each time the loop is executed until J reaches K, The next step 5, insert into the array in the the space just created. Finally the N is

incremented with 1 as new element is inserted.

(Inserting into a Linear Array)

INSERT (LA, N, K, ITEM) Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into the Kth position in LA.

Step 1 [Initialize counter.] Set $J := N$.

Step 2 Repeat Steps 3 and 4 while $J \geq K$.



**VIVEKANANDA GLOBAL
UNIVERSITY, JAIPUR**

(Established by Act 11/2012 of Rajasthan Govt. Covered u/s 2 (f) of UGC Act, 1956)

**NAAC
ACCREDITED
A+**
UNIVERSITY

Step 3 [Move Jth element downward.] Set $LA[J+1] := LA[J]$.

Step 4 [Decrease counter.] Set $J := J-1$.

[End of Step 2 loop.]

Step 5 [Insert element.] Set $LA[K] := \text{ITEM}$

Step 6 [Reset N.] Set $N := N+1$.

Step 7 Exit.

Algorithms for Deletion

(Deleting from a Linear Array)

DELETE(LA, N, K, ITEM). Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm deletes the Kth element from LA.

Step 1 Set $\text{ITEM} := LA[K]$.

Step 2 Repeat for $J = K$ to $N-1$:

[Move J + 1st element upward.] Set $LA[J] := LA[J+1]$.

[End of loop.]

Step 3 [Reset the number N of elements in LA.] Set $N := N-1$.

Step 4 Exit.



**VIVEKANANDA GLOBAL
UNIVERSITY, JAIPUR**

(Established by Act 11/2012 of Rajasthan Govt. Covered u/s 2 (f) of UGC Act, 1956)

**NAAC
ACCREDITED
A+
UNIVERSITY**

Multidimensional Array

The array discussed till now is also called as one-dimensional array, since each element in the array is referenced by a single subscript. Most programming language allow two- dimensional and three dimensional array i.e. where elements are referenced, respectively by two and three subscripts; these dimension may go upto as high as 7.

Two-Dimensional Array

A two-dimensional array, also known as a matrix, is a data structure that stores elements in a grid like format with rows and columns. Each element in the array is identified by a pair of indices: the row index and the column index.

In most programming languages, a two-dimensional array is implemented as an array of arrays. The outer array represents the rows, and each element of the outer array is an inner array that represents the columns.

A two-dimensional array $m \times n$; consists of $m.n$ data elements such that each element is specified by a pair of integers such as I, J called as subscript with following property.

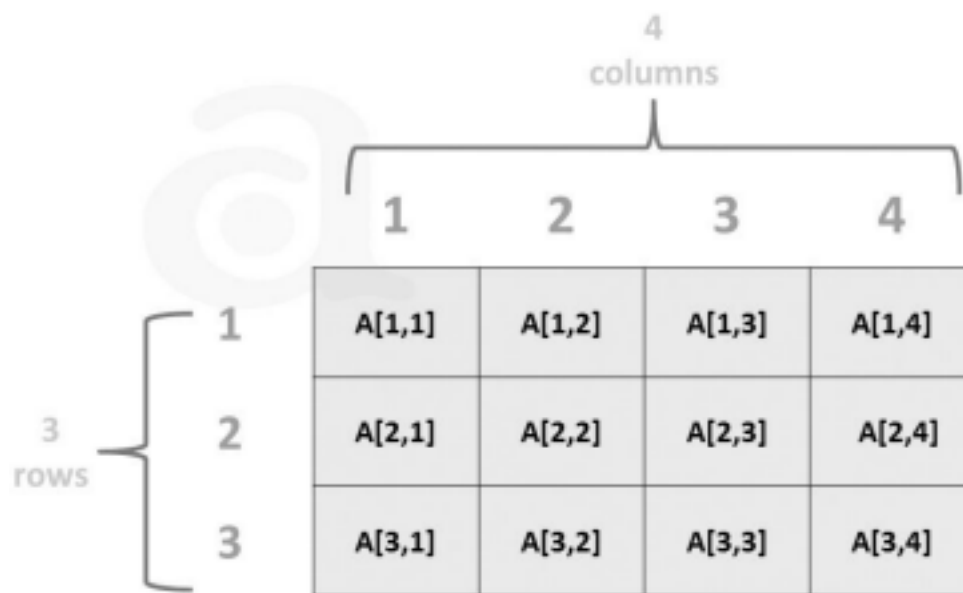
$$1 \leq I \leq m \text{ and } 1 \leq J \leq n$$

The element of array now can be seen as

$A[I][J]$ or $A_{I,J}$

Some time these two-dimensional arrays are also called as matrix array due to concept of matrix in mathematics and table in business applications.

Example, A 2D array can be seen in following figure with three (m) Rows and Four (n) Columns and each element is positioned at I,J th location i.e. $A[I][J]$ where I and J belongs to ROW and COLUMN respectively. For eg. $A[1][3]$ means element positioned at 1st ROW and 3rd Column.



The diagram illustrates a 2D array with 3 rows and 4 columns. The rows are indexed 1 to 3, and the columns are indexed 1 to 4. The elements are represented as $A[I][J]$ where I is the row index and J is the column index.

	1	2	3	4
1	$A[1,1]$	$A[1,2]$	$A[1,3]$	$A[1,4]$
2	$A[2,1]$	$A[2,2]$	$A[2,3]$	$A[2,4]$
3	$A[3,1]$	$A[3,2]$	$A[3,3]$	$A[3,4]$

Real life Example of 2D array

Take an example of result 2D array with snapshot of 7 students of 50 student class.

	Subject(Max.Marks)					
Student	Maths (150)	Chemistry (130)	Physics (120)	Geography (100)	History (60)	Computer Science (40)
A	90	50	90	60	70	80
B	100	80	80	40	80	70
C	90	60	70	70	90	70
D	80	65	80	80	60	60
E	80	65	85	95	50	90
F	70	75	65	85	40	60
G	65	35	50	77	80	80

Suppose in a class of 50 students each student is having marks in different subjects as given in following diagram. Assuming the students are named as A,B,C,D,E,F,G and subjects are names as Maths, Chemistry, Physics, Geography, History and Computer Science. In total 50 Rows and 6 Columns; The total entry in this table can be assigned is

ROWS X COLUMNS

i.e. $50 \times 6 = 300$

So, there will 300 marks entry for total 50 students for the above-named subjects in that class.

Suppose A is a two-dimensional $m \times n$ array.

The first dimension of A contains the index set $1, \dots, m$, with lower bound 1 and upper bound m ; and the second dimension of A contains the index set $1, 2, \dots, n$, with lower bound 1 and upper bound n .

The length of a dimension is the number of integers in its index set. The pair of lengths $m \times n$ (read "m by n") is called the size of the array. Some programming languages allow one to define multidimensional arrays in which the lower bounds are not 1. (Such arrays are sometimes called



**VIVEKANANDA GLOBAL
UNIVERSITY, JAIPUR**

(Established by Act 11/2012 of Rajasthan Govt. Covered u/s 2 (f) of UGC Act, 1956)

**NAAC
ACCREDITED
A+
UNIVERSITY**

non-regular.) However, the index set for each dimension still consists of the consecutive integers from the lower bound to the upper bound of the dimension. The length of a given dimension (i.e., the number of integers in its index set) can be obtained from the formula

$$\text{Length} = \text{upper bound} - \text{lower bound} + 1$$

Memory Representation of Multidimensional Array

In memory, a multidimensional array is typically represented as a contiguous block of memory. Although array is pictured as rectangular array of elements with m rows and n columns, the array in memory is represented by a block of m.n sequential memory locations.

The arrangement of the elements can be in row-major or column-major order, means the elements are laid out sequentially, row by row or column by column.

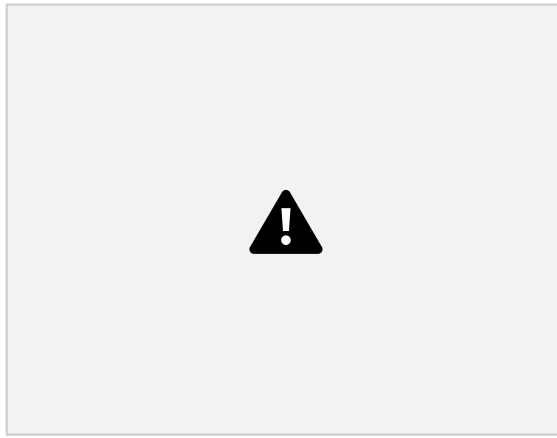
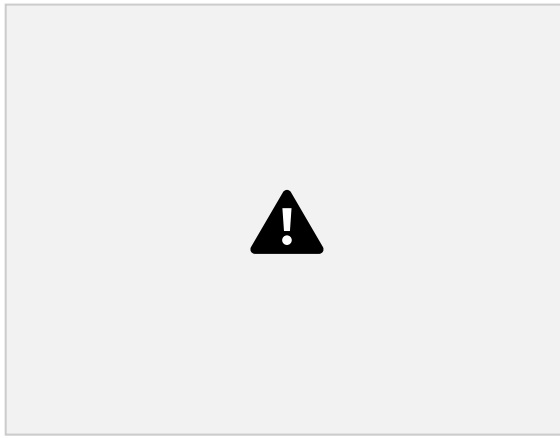
Row-Major Order:

In row-major order, the elements of each row are stored contiguously, and rows are stored one after another. This means that consecutive elements of the same row are stored together. Means elements are stored as 1st row, 2nd row and so on.

Column-Major Order:

In column-major order, the elements of each column are stored contiguously, and columns are stored one after another. This means that consecutive elements of the same column are stored together. Means elements are stored as 1st column, 2nd column and so on.



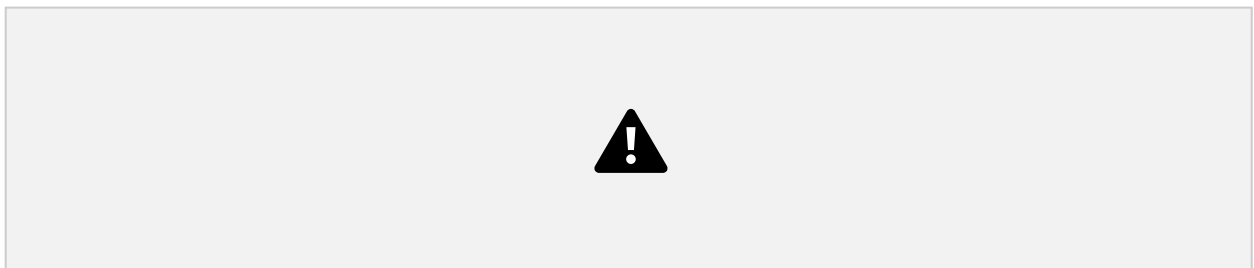


So, in general an example of matrix A (of size 3 X 3) can be seen as:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

The memory representation of the array in row-major order would be: 1 2 3 4 5 6 7 8 9. In this representation, the elements of the first row come first, followed by the elements of the second row, and so on.

For the same 3x3 matrix in column-major order, the memory representation would be: 1 4 7 2 5 8 3 6 9. In this representation, the elements of the first column come first, followed by the elements of the second column, and so on.





The choice of row-major or column-major order depends on the programming language or the specific requirements of the application. Most programming languages, including C and C++, use row-major order for multidimensional arrays.

However, it's important to note that some languages and libraries may use column-major order, such as Fortran and MATLAB.



Address calculation in Multidimensional Array

As we have seen the address calculation in linear array consists of the

$$\text{LOC (LA[K])} = \text{Base Address (LA)} + w (\text{K}-1)$$

Where Base Address – Address of First Element in the array LA

W words per memory

LOC (LA[K]) – Address of Kth Element in the array LA.

A similar situation can be seen in 2D array of M X N Dimension also. Means Computer tracks the Base address of Array A i.e. Address of First Element of Array A or we can say address of A[1][1]. And Calculate the address **LOC (A[I][J])** i.e. **address of A[I][J]** as

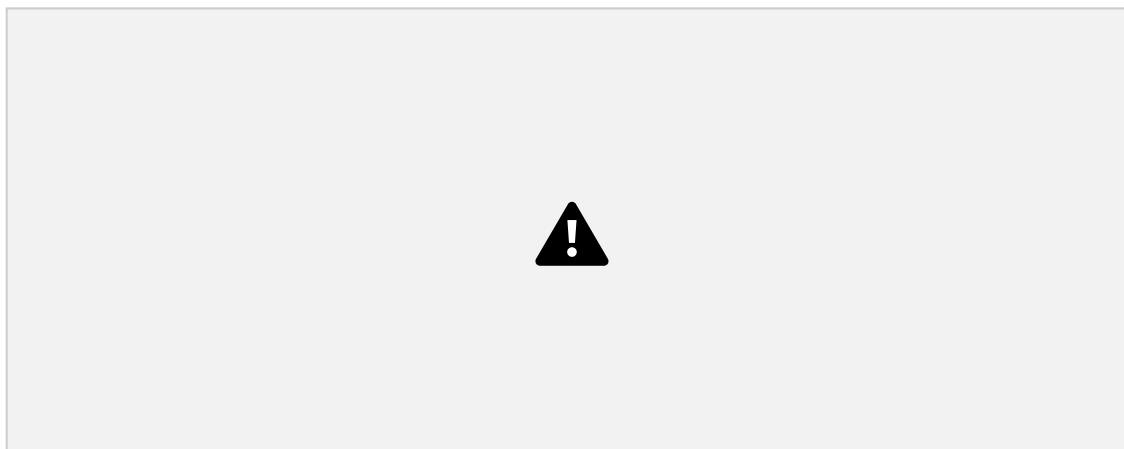
For Column Major Order

$$\text{LOC (A [I] [J])} = \text{Base Address (A)} + w [M(\text{J}-1) + (\text{I}-1)]$$

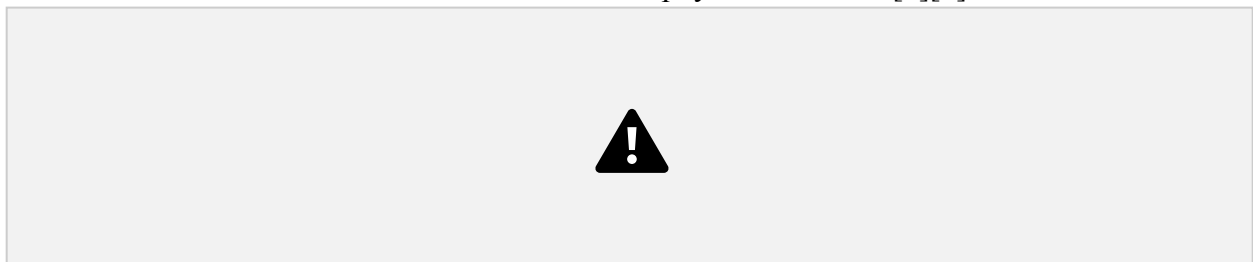
For ROW Major Order

$$\text{LOC (A [I] [J])} = \text{Base Address (A)} + w [N(\text{I}-1) + (\text{J}-1)]$$

Let's go back to our result example



Suppose the base address of result matrix is 200 and w = 4; Given m = 7 and n = 6 we want to calculate the **address** of student c marks in physics i.e. result [3][3]



For Column Major Order

$$\text{LOC [3][3]} = 200 + 4 [7(3-1)+(3-1)]$$

$$= 200 + 4 [14 + 2]$$

$$= 200 + 64$$

$$= 2064$$

For Row Major Order

$$\text{LOC}[3][3] = 200 + 4 [6(3-1) + (3-1)]$$

$$= 200 + 4 [12 + 2]$$

$$= 200 + 56$$

$$= 2056$$

General Multi-Dimensional Array

A multi-dimensional array is an array with more than two dimensions. While a two-dimensional array is like a grid with rows and columns, a multi-dimensional array extends this concept to include additional dimensions.

For example, a three-dimensional array can be visualized as a cube with multiple layers, rows, and columns. Each element in the array is identified by three indices: the first index represents the layer, the second index represents the row, and the third index represents the column.

In general Multi-Dimensional a Array B with n-dimensions can be seen as

$$M_1 \times M_2 \times M_3 \dots \times M_n$$

The Array is a collection of $m_1, m_2, m_3, \dots, m_n$ data elements in which each element specified by a list of n integers such that $K_1, K_2, K_3, \dots, K_n$ called subscript with property that

$$1 \leq K_1 \leq m_1 \quad 1 \leq K_2 \leq m_2 \quad 1 \leq K_3 \leq m_3 \quad 1 \leq K_n \leq m_n$$



The element of B with subscript $K_1, K_2, K_3, \dots, K_n$ will be denoted by

$$B[K_1, K_2, K_3, \dots, K_n]$$

For example A 3-D array, for example, uses three subscripts. The first two are just like a matrix, but the third dimension represents pages or sheets of elements.



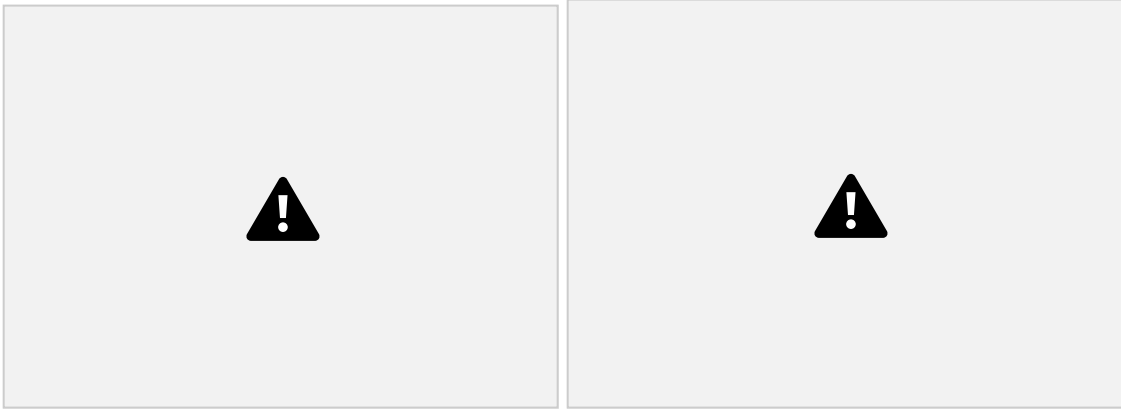
Sparse Matrices

A sparse matrix is a matrix that contains high portion of zero values. In contrast to dense matrices, which store all elements regardless of their value, sparse matrices only store the non-zero elements and their corresponding indices.

Sparse matrices are commonly encountered in various domains, including graph theory, scientific computing, and machine learning, where many real-world datasets have a large number of zero elements.

Sparse Matrices are categorised into Triangular and Tri-diagonal Matrices. The difference can be seen in the picture below





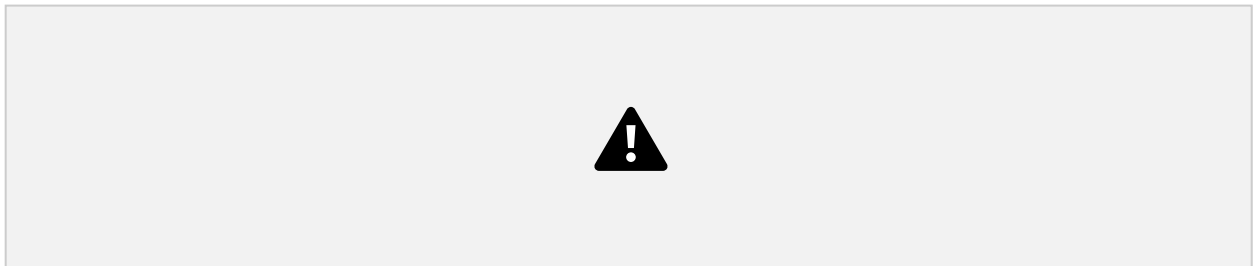
Triangular Matrix (Lower) Tri-Diagonal Matrix

Triangular Matrix – non-zeros elements are placed either below diagonal or upper diagonal. As seen in the picture; Its a Triangular Matrix (Lower) where all non-zero values are placed below diagonal in other case the **Triangular Matrix (upper)** will be having all non-zero values at upper part of the diagonal.

Tri-diagonal Matrices - non-zeros elements are placed at diagonal range. As seen in the picture; Its a Tri-diagonal Matrix where all non-zero values are placed at diagonal.

Why use a sparse matrix rather than a simple matrix?

- ✓ Storage: Because there are fewer non-zero items than zeros, less memory may be utilised to store solely those.
- ✓ Computing time: It can be reduced by rationally constructing a data structure that only traverses non-zero items.



The usage of a 2D array to represent a sparse matrix wastes a lot of resources since the zeroes in the matrix are rarely used. As a result, **we only save non-zero items instead of zeroes**. This entails using triples to store non-zero items (Row, Column, value).